SAMPLE-EFFICIENT REINFORCEMENT LEARNING FOR UAV COMMUNICATION SYSTEMS THROUGH CONSERVATIVE Q-LEARNING AND STATE COMPRESSION

Submitted by:

VISWAK RAMAMOORTHY BALAJI

Supervisor:

PROF. HOLGER CLAUSSEN

DR. JOSEANNE VIANA

Second Reader:

PROF. ANDREA VISENTIN

MSc Data Science and Analytics

School of Computer Science & Information Technology
University College, Cork

August 29, 2025

Abstract

Reinforcement learning (RL) has achieved major successes in simulation but remains constrained by poor sample efficiency, limiting its potential for real-world deployment. This thesis investigates methods to improve sample efficiency in continuous control, using a continuous control environment as a benchmark. A baseline is first established with the Soft Actor-Critic (SAC) algorithm in the online setting, where the agent learns directly through interaction. The resulting trajectories are then reused to construct offline datasets, providing a consistent foundation for further experimentation without requiring additional interaction. Building on this, Conservative Q-Learning (CQL) is implemented by applying a conservative penalty on Q-values and evaluated on an unbiased offline dataset with diverse trajectories. To further enhance efficiency, a Variational Autoencoder (VAE) is trained to compress state representations into compact latent vectors, which are then used for offline CQL training. The study demonstrates that CQL reduces the number of environment interactions required to solve the task by approximately 75% compared to online SAC, with reductions ranging from 40-75% depending on dataset conditions. Furthermore, combining CQL with VAE compression achieves an additional 14% minimum gain in sample efficiency, successfully solving the task using compressed states while maintaining performance. The proposed methods are developed and validated in a continuous control benchmark, which can be extended and applied to Unmanned Aerial Vehicle (UAV) communication systems, where energy, safety, and data collection constraints make sample-efficient reinforcement learning essential.

Declaration

I confirm that, except where indicated through the proper use of citations and references, this is my original work and that I have not submitted it for any other course or degree.

Signed: _____

Viswak Ramamoorthy Balaji
August 29, 2025

Acknowledgements

I would like to sincerely thank my supervisors, Prof. Holger Claussen and Dr. Joseanne Viana, for their guidance and support throughout this work. Dr. Joseanne's mentorship was invaluable in the completion of this thesis. A final thanks goes to my partner and my family who have supported me throughout the MSc programme.

Contents

List of Tables

List of Figures

Chapter 1

Introduction

Reinforcement Learning (RL) is a computational approach in which an agent learns to act optimally in an environment by maximizing cumulative rewards through trial-and-error interaction [1, 2]. Over the past decade, RL has achieved remarkable success in complex, high-dimensional control tasks, ranging from Atari video games [3] to continuous robotic locomotion [4, 5]. These achievements have been driven by advances in function approximation using deep neural networks, improved exploration strategies, and the availability of large-scale simulation environments such as OpenAI Gym [6] and DeepMind Control Suite [7].

Modern RL algorithms remain sample inefficient: they typically require millions of environment interactions to learn a stable policy [8]. While such data requirements are acceptable in simulation, they pose a severe barrier to real-world deployment, where data collection is expensive, slow, or potentially unsafe. This issue is particularly pronounced in UAV path planning, which must simultaneously optimize multiple objectives: obstacle avoidance, communication reliability, energy management, and operational security [9], [10], [11] within complex environments. Real-world UAV operations cannot support extensive trial-and-error learning due to hardware degradation, limited battery life, and safety constraints, making sample-efficient learning methods crucial for practical deployment [12].

Deep Reinforcement Learning (DRL) has demonstrated considerable success in UAV path planning applications including logistics delivery, security monitoring, and environmental surveillance [13], [14]. However, the sample inefficiency of traditional DRL algorithms remains problematic, as each real-world interaction incurs costs through equipment and operational expenses. Integration of telecommunication requirements to ensure network coverage and minimize connectivity gaps further compounds this complexity. Accurate wireless coverage assessment requires extensive data collection including detailed terrain maps, user distributions, base station locations, and related

parameters with requirements varying significantly across different geographical regions and infrastructure configurations. [15]. Similar constraints arise in autonomous driving [16], and industrial process control [17], where online trial-and-error learning is often infeasible. This has motivated research into algorithms and architectures that can learn effectively from limited data and make efficient use of every collected information available at each interaction with the environment.

One avenue for addressing this limitation is Offline Reinforcement Learning (Offline RL), also known as batch RL. In this setting, the agent learns entirely from a fixed dataset collected by one or more behavior policies, without further environment interaction [18]. Offline RL enables the use of pre-collected data, such as operational logs or expert demonstrations, to train policies without incurring the cost or risk of exploration. This paradigm has gained traction in recent years due to its potential to unlock vast stores of previously collected interaction data in domains like robotics [19], autonomous driving [20]. However, applying standard off-policy algorithms directly in the offline dataset setting can lead to poor performance due to distributional shift between the behavior policy that generated the dataset and the learned policy [21, 22]. This mismatch often results in Q-function overestimation for out-of-distribution (OOD) actions [23, 24], which the agent may then exploit, leading to unstable learning and poor generalization.

To address these challenges, several approaches have been proposed. Behavior Cloning (BC) [25] constrains the learned policy to imitate the behavior policy but sacrifices potential improvements beyond that policy. Batch-Constrained Q-Learning (BCQ) [21] and Bootstrapping Error Accumulation Reduction (BEAR) [26] explicitly restrict policy updates to remain close to the behavior policy distribution, thereby reducing extrapolation error. More recently, Conservative Q-Learning (CQL) [22] augments the critic loss with a conservative penalty, explicitly reducing Q-values for actions outside the dataset distribution. This modification addresses the overestimation problem and has achieved state-of-the-art results on several offline RL benchmarks [27]. Other strategies, such as uncertainty-aware exploration [28], model-based offline RL [29], and more recent methods such as Implicit Q-Learning (IQL) [30], further attempt to regularize learning in the presence of distributional shift. Surveys such as [31, 32] provide comprehensive overviews of recent progress, while recent studies [33] highlight the importance of dataset diversity for generalization in offline RL.

Another complementary direction for improving sample efficiency involves representation learning [34, 35]. By encoding high-dimensional states into compact latent representations, agents can potentially learn more efficiently and generalize better to unseen states. Variational Autoencoders (VAEs) [36] are a widely used tool for such compression, and have been integrated into model-based RL frameworks like Dreamer [37] to enable long-horizon planning in latent space. In offline RL, latent state representations can reduce input complexity, remove redundancy, and potentially improve generalization, particularly in environments with high-dimensional observations such as images or sensor arrays [38]. While the benefits of representation learning are well-

established in high-dimensional domains, its role in more compact state spaces remains less explored. This motivates our integration of representation learning into offline reinforcement learning in a continuous control task with an eight-dimensional state space.

We begin by establishing a baseline with the Soft Actor-Critic (SAC) algorithm [5] in both online and offline training regimes. The online setting, where the agent interacts with the environment at each step, serves as a reference for achievable performance when exploration is unconstrained. In contrast, the offline setting highlights the challenges of learning solely from fixed datasets, where SAC typically suffers from Q-value overestimation and unstable convergence, as noted in prior work [21, 22].

To extend this baseline, we implement Conservative Q-Learning (CQL) [22] on top of SAC and evaluate it using an unbiased offline dataset composed of diverse trajectories. This dataset is generated using multiple behaviour policies to provide broad state-action coverage, enabling systematic study of CQL under conditions that better approximate real-world offline learning. Finally, we integrate representation learning into the offline pipeline by training a Variational Autoencoder (VAE) [36] and combining it with CQL. The VAE encodes the offline dataset into lower-dimensional latent representations, effectively reducing the state space before training CQL. This approach investigates whether representation learning can further enhance sample efficiency by focusing learning on compressed but informative features of the environment.

The primary aim of this thesis is to improve sample efficiency in reinforcement learning by systematically evaluating SAC, CQL, and the integration of VAE with CQL in a continuous control task. While the experiments are conducted in a simulated environment, the methods are to be transferable to UAVs, where interaction is costly or hazardous, contributing to the broader development of sample-efficient reinforcement learning methods.

This thesis is organized into five chapters. Chapter 2 establishes the theoretical foundation, covering reinforcement learning fundamentals, Markov Decision Processes, continuous control methods, and the main algorithms studied in this work, namely Soft Actor-Critic (SAC), Conservative Q-Learning (CQL), and the integration of a Variational Autoencoder (VAE) with CQL. Chapter 3 describes the methodology, including the architecture overview, experimental environment, dataset generation, algorithm implementations, hyperparameter configuration, and the training and evaluation flow. Chapter 4 presents the experimental results across the methods on different datasets, followed by a detailed discussion of sample efficiency, dataset quality, state compression trade-offs, and identified challenges and outlines directions for future work. Finally, Chapter 5 concludes the thesis.

Chapter 2

Theoretical Background

This chapter presents a comprehensive theoretical overview of the principles and methods underpinning the research in improving sample efficiency within DRL , specifically for continuous control tasks. We begin by providing detailed explanations of fundamental DRL concepts, followed by a clear introduction to Markov Decision Processes (MDPs) and the core value functions that drive agent decision-making.

2.1 Reinforcement Learning Framework

Reinforcement Learning (RL) provides a framework for sequential decision-making, where an agent interacts with an environment in discrete time steps to maximize cumulative rewards [1]. In small or well-structured problems, the agentâs behavior can be described exactly using tables or simple linear functions to represent value functions and policies. However, as soon as the state or action space becomes large, continuous, or high-dimensional, such representations quickly become impractical. Deep Reinforcement Learning (DRL) addresses this by using deep neural networks to approximate the policy $\pi_\theta(a|s)$, the value function $V_\phi(s)$, or the action-value function $Q_\phi(s,a)$ [3]. The problem is commonly modeled as a Markov Decision Process (MDP), defined by the tuple $(\mathscr{S},\mathscr{A},P,R,\gamma)$, where:

- $\mathscr{S}$ is the set of possible states $s \in \mathscr{S}$.
- $\mathscr{A}$ is the set of possible actions $a \in \mathscr{A}$.
- $P(s'|s,a)$ is the state transition probability, giving the likelihood of moving to state s' when action a is taken in state s.
- $R(s,a)$ is the reward function, specifying the immediate scalar feedback $R \in \mathbb{R}$.

- $\gamma \in [0,1]$ is the discount factor, weighting future rewards.

At each time step t, the agent observes a state s_t, selects an action a_t according to a policy π, receives a reward $r_t = R(s_t, a_t)$, and transitions to a new state $s_{t+1} \sim P(\cdot|s_t, a_t)$. The goal is to learn a policy $\pi(a|s)$ that maximizes the expected return:

$$J(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty}\gamma^t r_t\right], \tag{2.1}$$

where $\tau = (s_0, a_0, r_0, s_1, a_1, ...)$ denotes a trajectory generated by π.

Two key value functions are used in RL:

$$V^\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty}\gamma^t r_t \,|\, s_0 = s\right], \tag{2.2}$$

$$Q^\pi(s,a) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty}\gamma^t r_t \,|\, s_0 = s, a_0 = a\right]. \tag{2.3}$$

These satisfy the Bellman equations [39]:

$$V^\pi(s) = \mathbb{E}_{a\sim\pi}\left[R(s,a) + \gamma\,\mathbb{E}_{s'\sim P}\left[V^\pi(s')\right]\right], \tag{2.4}$$

$$Q^\pi(s,a) = R(s,a) + \gamma\,\mathbb{E}_{s'\sim P,\,a'\sim\pi}\left[Q^\pi(s',a')\right]. \tag{2.5}$$

The optimal value functions, $V^*(s)$ and $Q^*(s,a)$, satisfy the Bellman optimality equation:

$$Q^*(s,a) = R(s,a) + \gamma\,\mathbb{E}_{s'\sim P}\left[\max_{a'} Q^*(s',a')\right], \tag{2.6}$$

and the corresponding optimal policy is:

$$\pi^*(s) = \arg\max_a Q^*(s,a). \tag{2.7}$$

DRL algorithms can be broadly categorized into:

- Value-based methods, which estimate $Q(s,a)$ and derive a policy by acting greedily w.r.t. Q (e.g., DQN [3]).
- Policy-based methods, which directly optimize $J(\pi)$ via policy gradient techniques [40].
- Actor-Critic methods, which combine both, using a parameterized actor π_θ and critic Q_ϕ [41].

In continuous action spaces, value-based methods become impractical due to

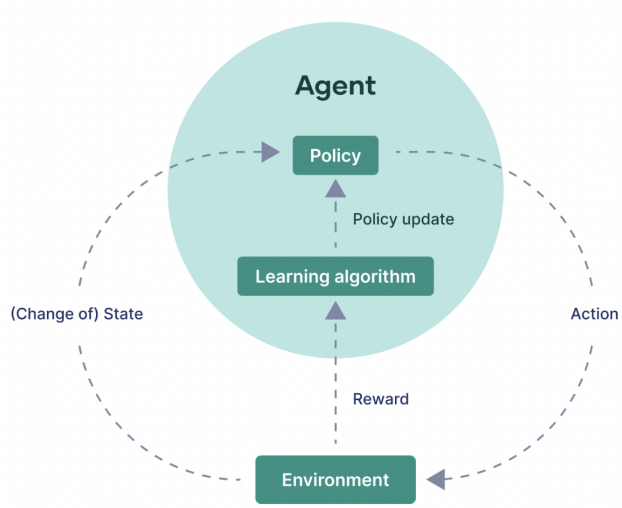


Figure 2.1: Inside the agent: policy, learning algorithm, and their interaction with the environment.

the $\max_{a'} Q(s', a')$ term requiring continuous optimization at each step. Actor-critic methods, such as Soft Actor-Critic [5], are better suited for such domains.

2.2 Markov Decision Processes

An MDP provides a mathematical model for sequential decision-making. Formally, an MDP is defined as :

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma) \tag{2.8}$$

The *Markov property* assumes that the future state depends only on the current state and action, and not on the history of previous states or actions:

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0) = P(s_{t+1} \mid s_t, a_t). \tag{2.9}$$

The agent's objective is to learn a policy $\pi(a|s)$, which is a mapping from states to a probability distribution over actions, that maximizes the expected return:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]. \tag{2.10}$$

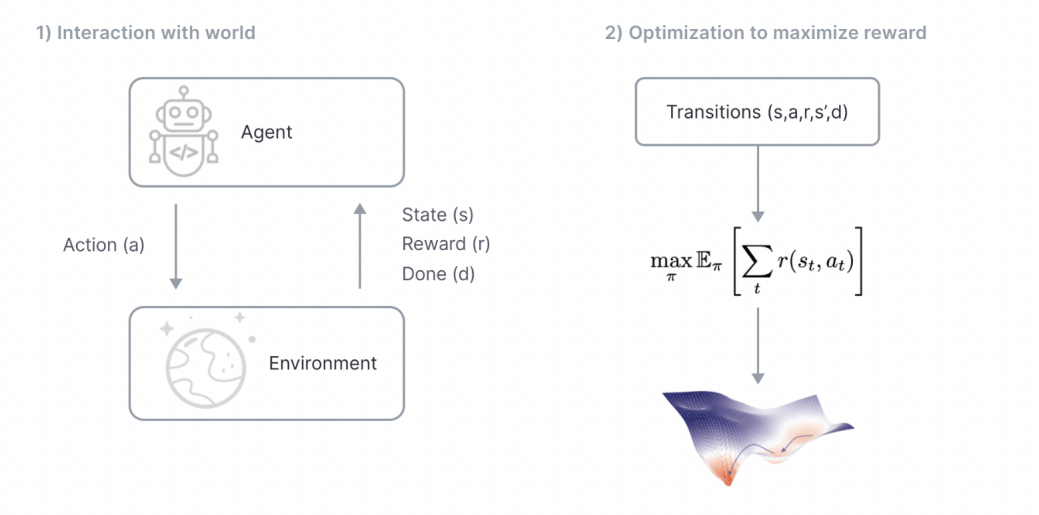


Figure 2.2: (Left) Agent–environment interaction signals; (Right) the optimization objective to maximize expected return $J(\pi)$.

To facilitate learning, RL algorithms often use two related value functions:

- The *state-value function* $V^\pi(s)$, which estimates the expected return starting from state s and following policy π thereafter:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \,\middle|\, s_0 = s \right]. \tag{2.11}$$

- The *action-value function* (Q-function) $Q^\pi(s,a)$, which estimates the expected return starting from state s, taking action a, and following π thereafter:

$$Q^\pi(s,a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \,\middle|\, s_0 = s, a_0 = a \right]. \tag{2.12}$$

These value functions satisfy recursive relationships known as the Bellman equations [39]:

$$V^\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s),\, s'\sim P} \left[R(s,a) + \gamma V^\pi(s') \right], \tag{2.13}$$

$$Q^\pi(s,a) = \mathbb{E}_{s'\sim P} \left[R(s,a) + \gamma \mathbb{E}_{a'\sim\pi(\cdot|s')} Q^\pi(s',a') \right]. \tag{2.14}$$

In the special case of the *optimal policy* π^*, the corresponding value functions $V^*(s)$ and $Q^*(s,a)$ satisfy the Bellman optimality equations:

$$V^*(s) = \max_{a\in\mathcal{A}} \mathbb{E}_{s'\sim P} \left[R(s,a) + \gamma V^*(s') \right], \tag{2.15}$$

$$Q^*(s,a) = \mathbb{E}_{s' \sim P}\left[R(s,a) + \gamma \max_{a' \in \mathscr{A}} Q^*(s',a')\right]. \tag{2.16}$$

In discrete environments, $\mathscr{S}$ and $\mathscr{A}$ are finite sets, and these equations can be solved exactly in principle. However, in continuous or high-dimensional domains, explicit enumeration is infeasible, requiring function approximation methods such as deep neural networks to estimate V^π and Q^π. This is particularly important for continuous control tasks, which are the main focus of this thesis.

2.3 Continuous Control in Reinforcement Learning

Continuous control tasks involve action spaces $\mathscr{A} \subset \mathbb{R}^n$, where actions are real-valued vectors rather than discrete labels [4]. These problems are common in robotics, locomotion, manipulation, and autonomous vehicle control, where control signals (e.g., torques, velocities, steering angles) vary smoothly over time.

The primary challenges in continuous control include:

- Exploration: Random exploration is inefficient; effective strategies must balance coverage of the action space with focusing on promising actions [5].
- Optimization: Directly finding $\arg\max_a Q(s,a)$ is non-trivial when a is continuous and high-dimensional.
- Stability: Small changes in actions can cause large changes in returns, making policy learning sensitive to noise.

Policy gradient methods [40] parameterize the policy $\pi_\theta(a|s)$ and update parameters via:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{s \sim d^\pi, a \sim \pi_\theta}\left[\nabla_\theta \log \pi_\theta(a|s) Q^\pi(s,a)\right], \tag{2.17}$$

where d^π is the state distribution under π. This avoids the $\max_a$ operation entirely, making them well-suited to continuous domains.

Modern continuous control algorithms, such as DDPG [4], TD3 [42], and SAC [5], use deterministic or stochastic policies in an actor-critic framework to achieve sample-efficient learning in continuous action spaces.

2.4 Soft Actor Critic (SAC)

Soft Actor Critic (SAC) [5] is an off-policy actor-critic algorithm designed for continuous action spaces that combines maximum entropy reinforcement learning with the stability

benefits of off-policy training. The key idea is to augment the standard expected return objective with an entropy term, encouraging exploration by favoring policies with higher entropy.

The entropy of a stochastic policy $\pi(a|s)$ is defined as:

$$\mathscr{H}(\pi(\cdot|s)) = -\mathbb{E}_{a\sim\pi(\cdot|s)}\left[\log\pi(a|s)\right]. \tag{2.18}$$

The maximum entropy objective is:

$$J(\pi) = \sum_{t=0}^{\infty} \mathbb{E}_{(s_t,a_t)\sim\rho_\pi}\left[R(s_t,a_t) + \alpha\,\mathscr{H}(\pi(\cdot|s_t))\right], \tag{2.19}$$

where $\alpha > 0$ is the temperature parameter controlling the trade-off between reward maximization and entropy maximization. A higher α promotes more exploration, while a lower α focuses on exploitation.

The corresponding soft state value and soft Q-value functions are:

$$V^\pi(s) = \mathbb{E}_{a\sim\pi}\left[Q^\pi(s,a) - \alpha\log\pi(a|s)\right], \tag{2.20}$$

$$Q^\pi(s,a) = R(s,a) + \gamma\,\mathbb{E}_{s'\sim P}\left[V^\pi(s')\right]. \tag{2.21}$$

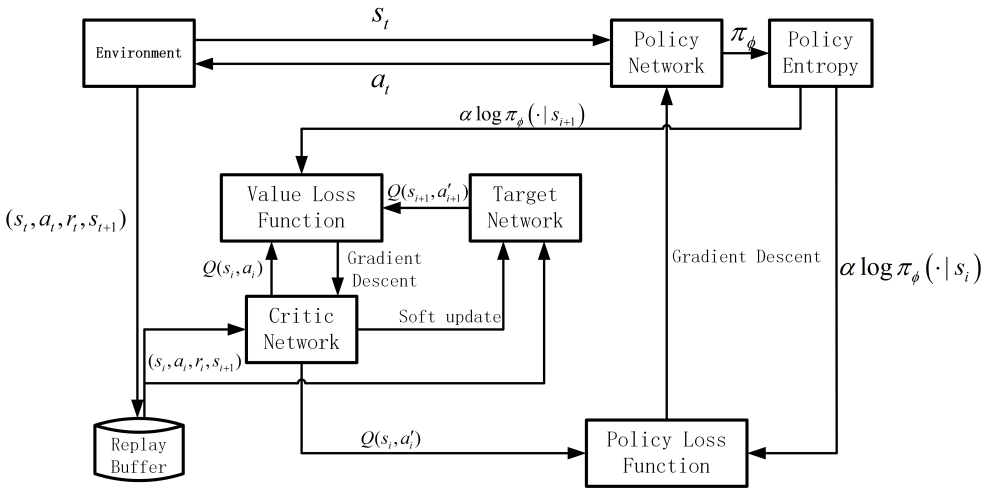


Figure 2.3: Soft Actor-Critic overview: stochastic policy (actor), twin Q critics, target network, entropy-regularized updates.

The soft Bellman backup for Q is:

$$Q(s,a) \leftarrow R(s,a) + \gamma\,\mathbb{E}_{s'\sim P,a'\sim\pi}\left[Q(s',a') - \alpha\log\pi(a'|s')\right]. \tag{2.22}$$

SAC maintains three sets of parameters:

- ϕ: parameters of the soft Q-function(s) $Q_\phi(s,a)$.
- θ: parameters of the stochastic policy $\pi_\theta(a|s)$.
- ψ: parameters of the value function $V_\psi(s)$ (optional in later versions of SAC).

The critic parameters ϕ are updated by minimizing the soft Bellman residual:

$$J_Q(\phi) = \mathbb{E}_{(s,a,r,s')\sim\mathscr{D}}\left[\frac{1}{2}\left(Q_\phi(s,a) - y\right)^2\right], \tag{2.23}$$

where the target is:

$$y = r + \gamma \mathbb{E}_{a'\sim\pi_\theta}\left[\min_{i\in\{1,2\}} Q_{\bar{\phi}_i}(s',a') - \alpha \log \pi_\theta(a'|s')\right], \tag{2.24}$$

and $Q_{\bar{\phi}}$ denotes a target network with delayed updates.

The actor parameters θ are updated to minimize:

$$J_\pi(\theta) = \mathbb{E}_{s\sim\mathscr{D},\, a\sim\pi_\theta}\left[\alpha \log \pi_\theta(a \mid s) - Q_\phi(s,a)\right], \quad \text{where } a = \mu_\theta(s) + \sigma_\theta(s)\odot\epsilon,\ \epsilon\sim\mathcal{N}(0,I). \tag{2.25}$$

which corresponds to maximizing both the expected Q-value and the entropy.

The temperature parameter α can be fixed or adjusted automatically [43] by minimizing:

$$J(\alpha) = \mathbb{E}_{a\sim\pi_\theta}\left[-\alpha\left(\log \pi_\theta(a|s) + \bar{\mathscr{H}}\right)\right], \tag{2.26}$$

where $\bar{\mathscr{H}}$ is a target entropy, often set to $-\dim(\mathscr{A})$ for continuous actions.

SAC's combination of off-policy data reuse, entropy regularization, and stability from target networks makes it one of the most sample-efficient algorithms for continuous control.

2.4.1 Online SAC

In the online reinforcement learning setting, the Soft Actor-Critic (SAC) algorithm operates by continuously interacting with the environment at each timestep. After executing an action a_t according to the current stochastic policy $\pi_\theta(a_t|s_t)$, the agent observes the next state s_{t+1} and the reward r_t, storing the transition (s_t, a_t, r_t, s_{t+1}) in a replay buffer $\mathscr{D}$. At each training step, SAC samples mini-batches from $\mathscr{D}$ to update both the policy and the Q-function estimators [5].

Online SAC benefits from the ability to adapt its exploration strategy based on the evolving policy and value function estimates. The maximum entropy objective

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}_{(s_t,a_t)\sim\rho_\pi} \left[R(s_t, a_t) + \alpha \mathscr{H}(\pi(\cdot|s_t)) \right]$$

encourages the policy to explore diverse actions, improving coverage of the state-action space. The temperature parameter α automatically adjusts to balance reward maximization with entropy, allowing SAC to remain robust in sparse-reward settings [5].

However, despite its stability and asymptotic performance advantages, online SAC is often sample inefficient, requiring a large number of interactions before converging to a high-performing policy [8]. This is particularly problematic in real-world applications, where interaction cost is high due to safety, wear-and-tear, or time constraints [12, 16].

2.4.2 Offline SAC

In the offline setting, SAC is trained using a fixed dataset $\mathscr{D} = \{(s_i, a_i, r_i, s'_i)\}_{i=1}^{N}$ collected by one or more behaviour policies $\beta(a|s)$. No further interaction with the environment is permitted during training [18]. While the algorithmic structure of SAC remains unchanged, the removal of online exploration introduces a critical challenge: distributional shift [21, 22].

The learned policy π_θ may propose actions a in states s that are poorly represented in $\mathscr{D}$. Since the Q-function is trained only on data from β, its estimates for such out-of-distribution (OOD) actions can be arbitrarily inaccurate. When combined with SAC's policy improvement step, this can lead to *Q-value overestimation* [23], where the agent exploits erroneous high-value estimates for unsupported actions, causing policy degradation.

Formally, if $\mathscr{S}_\beta \times \mathscr{A}_\beta$ denotes the state-action support of β, offline SAC may select $a \notin \mathscr{A}_\beta(s)$, where the Bellman backup

$$Q(s,a) \leftarrow R(s,a) + \gamma \mathbb{E}_{s'\sim P}\left[V(s') \right]$$

relies on extrapolated Q-values outside the dataset's distribution. This extrapolation error accumulates over training iterations [21], often leading to divergence. This results in compounded bootstrapping error, since successive targets depend on increasingly extrapolated Q-values outside the dataset's action support $\mathscr{A}_\beta(s)$.

Due to this instability, vanilla SAC is generally unsuitable for offline RL without additional regularization or constraints. This limitation has motivated the development of algorithms such as Batch-Constrained Q-Learning (BCQ) [21], Bootstrapping Error Accumulation Reduction (BEAR) [26], and Conservative Q-Learning (CQL) [22], which

explicitly address the OOD action problem in offline settings.

2.5 Conservative Q-Learning (CQL)

Conservative Q-Learning (CQL) [22] is an offline reinforcement learning algorithm designed to address the distributional shift and Q-value overestimation issues encountered in standard offline SAC. CQL modifies the Q-function update by introducing a penalty term that lowers the estimated values of out-of-distribution (OOD) actions, thereby encouraging the learned policy to remain within the support of the dataset.

In standard SAC, the critic parameters ϕ are updated by minimizing the Bellman residual:

$$\mathscr{L}_{\text{SAC}}(\phi) = \mathbb{E}_{(s,a,r,s')\sim\mathscr{D}}\left[\frac{1}{2}\left(Q_\phi(s,a) - \left(r + \gamma \mathbb{E}_{a'\sim\pi_\theta(\cdot|s')}\left[Q_{\bar{\phi}}(s',a') - \alpha \log \pi_\theta(a'|s')\right]\right)\right)^2\right],$$

where $Q_{\bar{\phi}}$ is a target Q-network and α is the entropy temperature.

CQL augments this with a conservative penalty term:

$$\mathscr{L}_{\text{CQL}}(\phi) = \mathscr{L}_{\text{SAC}}(\phi) + \alpha_{\text{cql}} \cdot \mathbb{E}_{s\sim\mathscr{D}}\left[\log\left(\mathbb{E}_{a\sim\mathscr{S}(s)}\exp(Q_\phi(s,a))\right) - \mathbb{E}_{a\sim\mathscr{D}(a|s)}Q_\phi(s,a)\right],$$

where α_{cql} controls the strength of the penalty. The first term inside the brackets estimates the log-sum-exp over actions (emphasizing potentially high OOD Q-values), while the second term represents the average Q-value for in-dataset actions. The difference encourages Q_ϕ to assign higher values only to actions supported by the dataset.

In practice, the log-sum-exp is computed using a finite set of sampled actions, which may include:

- Actions sampled uniformly from the action space (to capture extreme OOD cases).
- Actions sampled from the current policy π_θ (to regularize near-policy actions).
- Actions from the dataset $\mathscr{D}$ (for in-distribution anchoring).

By penalizing OOD action values, CQL mitigates overestimation and prevents the learned policy from exploiting erroneous Q-values. This approach provides a form of implicit behaviour regularization without explicitly constraining the policy's divergence from the behaviour policy, distinguishing it from methods such as BCQ [21] and BEAR [26].

Empirical results from the original CQL work [22] demonstrate substantial performance gains across a variety of offline RL benchmarks, particularly in tasks with narrow

or suboptimal datasets. CQL has since been adopted and extended in subsequent research [27, 44], with applications in robotics [19], autonomous driving [45], and recommender systems [46], making it one of the most widely used baselines in offline RL studies.

2.6 Variational Autoencoder Integration

While this thesis does not focus on representation learning as a standalone research topic, we employ a Variational Autoencoder (VAE) [36] as a practical tool to enhance the sample efficiency of Conservative Q-Learning (CQL) in an offline reinforcement learning setting. The motivation for this integration is to reduce state dimensionality, remove redundant information, and potentially improve generalization by focusing the learning process on the most relevant features.

A VAE consists of:

- An **encoder** $q_\phi(z|s)$, which maps the original state $s \in \mathbb{R}^n$ to a latent vector $z \in \mathbb{R}^m$ ($m < n$), parameterized by mean $\mu_\phi(s)$ and variance $\sigma_\phi(s)$.
- A **decoder** $p_\theta(s|z)$, which reconstructs the original state from its latent representation.

The training objective is to maximize the Evidence Lower Bound (ELBO):

$$\mathscr{L}_{\text{VAE}}(\theta, \phi; s) = \mathbb{E}_{q_\phi(z|s)}[\log p_\theta(s|z)] - \beta \cdot D_{\text{KL}}\left(q_\phi(z|s) \parallel p(z)\right),$$

where:

- The first term ensures accurate reconstruction of the state.
- The second term regularizes the latent space to follow a prior distribution $p(z)$, typically $\mathcal{N}(0, I)$.
- β is a hyperparameter controlling the trade-off between reconstruction fidelity and latent space regularization.

Sampling from $q_\phi(z|s)$ is done via the reparameterization trick:

$$z = \mu_\phi(s) + \sigma_\phi(s) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I),$$

allowing gradients to propagate through the stochastic latent variables.

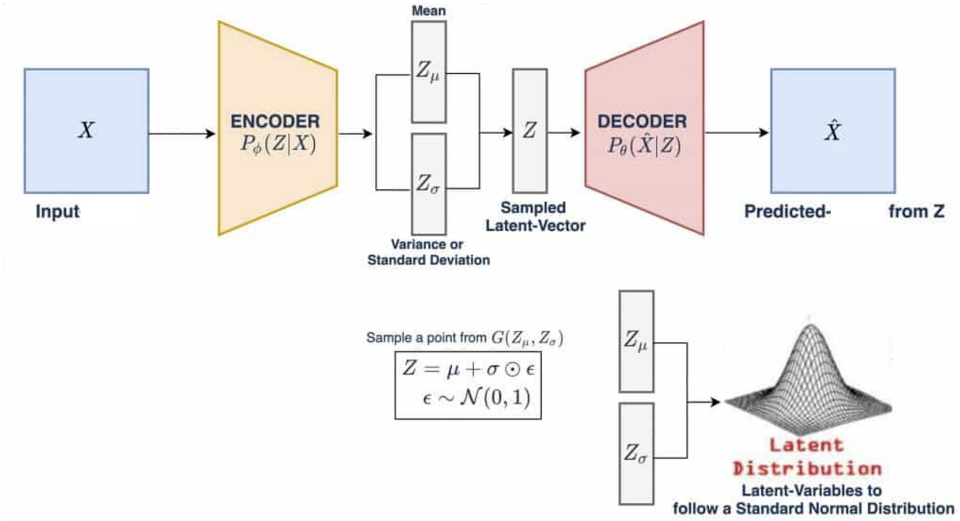


Figure 2.4: VAE used for state-space compression: encoder $q_\phi(z|s)$, latent z, and decoder $p_\theta(s|z)$ integrated with offline CQL.

In our integration, the encoder transforms the original LunarLanderContinuous-v2 state space into a compressed latent representation, which is then used as the input to CQL's policy and Q-networks. We also experiment with removing selected state variables before encoding to further test robustness to incomplete information. The hypothesis is that by reducing the dimensionality and noise in the input, CQL can achieve faster convergence and better stability, especially when the dataset size is limited.

This approach differs from prior uses of VAEs in reinforcement learning [37, 21, 38], where they are typically applied in high-dimensional observation domains such as images. Here, the VAE is applied to a low-dimensional, continuous control environment, specifically as a means to improve sample efficiency in offline RL through targeted state-space compression. In this thesis, the offline replay buffer is re-encoded into this latent space, and the resulting latent transitions are then used directly for training CQL, enabling a systematic evaluation of whether representation learning can enhance sample efficiency in offline RL.

Chapter 3

Methodology

This chapter provides an overview of the system architecture and the implementation approach adopted in this work. It describes how the core components of the reinforcement learning framework are structured and integrated, including the training pipeline, model architectures, and data handling. The goal is to outline how theoretical methods are translated into practical, working systems.

3.1 Architecture Overview

The overall system design for this thesis is illustrated in Figure 3.1. The architecture is modular, with clear separation between environment interaction, dataset generation, representation learning, and offline reinforcement learning. This modularity allows systematic experimentation with different pipelines under consistent conditions.

At the top of the workflow, the environment (`LunarLanderContinuous-v2`) produces trajectories, which are collected into replay buffers. To avoid bias from expert-only demonstrations, an unbiased offline dataset is generated using multiple behaviour policies, giving broader state-action coverage.

The system then diverges into two complementary pipelines:

- The VAE pipeline (purple box) trains a Variational Autoencoder on raw environment states, producing a compressed latent representation. This generates an encoded replay buffer that reduces input dimensionality while preserving essential information.
- The CQL pipeline (green box) implements offline training. It can operate on either the raw replay buffer or the VAE-encoded replay buffer, enabling direct

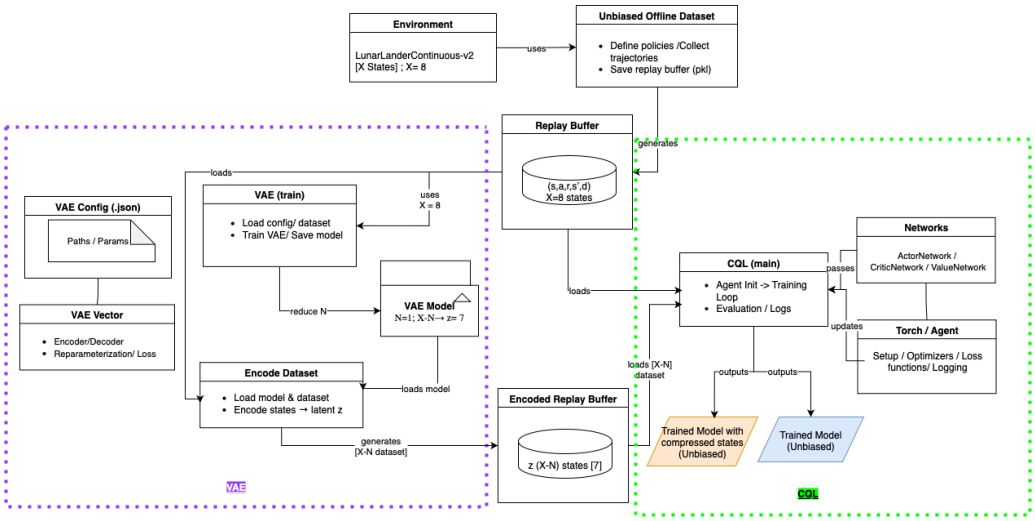


Figure 3.1: High-level workflow diagram. The **left (purple)** shows the VAE pipeline, where the replay buffer is used to train a VAE and generate an encoded replay buffer with compressed states. The **right (green)** shows the CQL pipeline, which can train on either raw or encoded datasets using actor, critic, and value networks. The outputs are two trained models: one using compressed states (orange) and one using raw states (blue).

comparison of learning in original versus compressed state spaces.

By structuring the system in this way, four experimental pipelines can be studied under consistent conditions: (1) online SAC, (2) offline SAC, (3) CQL, and (4) VAE+CQL. This separation makes it possible to isolate the effect of algorithmic changes (CQL) and representation learning (VAE) on sample efficiency.

3.2 Experimental Environment

All experiments were conducted in the `LunarLanderContinuous-v2` environment from OpenAI Gym [6]. The task simulates landing a spacecraft on a designated pad using a main engine and two side thrusters. The agent must balance thrust and orientation to achieve a soft touchdown, making the task a compact but non-trivial benchmark for continuous control.

State space

The environment provides an 8-dimensional state vector:

$$s = [x, y, \dot{x}, \dot{y}, \theta, \dot{\theta}, c_{\text{left}}, c_{\text{right}}], \tag{3.1}$$

where the components are defined in Table 3.1.

Table 3.1: LunarLanderContinuous-v2 state representation (8 dimensions).

Variable	Description
x, y	Position relative to landing pad
$\dot{x}, \dot{y}$	Horizontal and vertical velocities
θ	Orientation angle of the lander
$\dot{\theta}$	Angular velocity
$c_{\text{left}}, c_{\text{right}}$	Binary indicators for left/right leg contact with the ground

Action space

The action space is two-dimensional:

$$a = [a_{\text{main}}, a_{\text{side}}],$$

where $a_{\text{main}} \in [0, 1]$ controls the main engine throttle and $a_{\text{side}} \in [-1, 1]$ controls the side thrusters for lateral movement and rotation.

This environment provides a balance of realism and efficiency: it features continuous dynamics, delayed rewards, and sensitivity to control signals, yet remains lightweight enough for large-scale experimentation. Its compact 8-D state space and 2-D action space make it an effective platform for systematically studying sample efficiency in reinforcement learning.

3.3 Dataset and Replay Buffer

Reinforcement learning agents in this thesis train from replay buffers, which store environment interactions in the form of transitions (s, a, r, s', d), where s and s' are states, a is the action, r is the reward, and d is the terminal flag. Mini-batches are uniformly sampled from these buffers during training.

Three types of datasets were used:

Biased saved dataset

A replay buffer was obtained by saving trajectories from a previously trained SAC agent. Because this buffer is dominated by successful trajectories, it provides mostly high-return examples but limited exploration. Such biased datasets can be useful for warm-starting learning, but they restrict coverage of the state–action space.

Unbiased dataset

To create a more representative training set, we developed an UnbiasedDatasetGenerator that collects trajectories using multiple handcrafted behaviour policies. These include:

- **Random policy:** uniformly sampled actions.
- **Conservative policy:** stable, low-variance control.
- **Exploratory policy:** aggressively samples diverse actions.
- **Failing policy:** deliberately poor control to add negative examples.
- **Noisy expert and mixed policies:** near-expert behaviour with injected noise.

By combining trajectories from these policies in fixed proportions, the resulting dataset avoids over-representing optimal behaviour and offers broader state–action coverage. This unbiased dataset serves as the primary benchmark for offline SAC, CQL, and VAE+CQL experiments.

Encoded dataset

For representation-learning experiments, the unbiased dataset is further processed using a Variational Autoencoder (VAE). The six continuous state variables are encoded into a latent vector, while the two binary leg-contact indicators are appended unchanged. The resulting 7-D states form an encoded replay buffer, which is used directly in offline CQL training. This provides a consistent way to evaluate how state compression affects sample efficiency.

3.4 Algorithm Implementations

This section describes the core algorithmic components used in this thesis. The implementation is modular, with separate files handling neural network architectures, training loops, and dataset management. The four main blocks are: SAC, CQL, the VAE module, and the integration of VAE with CQL.

3.4.1 Soft Actor-Critic (SAC)

SAC is implemented in `sac_online.py` (online variant) and `sac_offline.py` (offline variant). The networks are defined in `networks.py`, including:

- an actor network producing a Tanh-squashed Gaussian policy using the reparameterization trick,
- two independent Q-networks (twin critics),
- and a target critic for stable value estimation.

The training loop handles buffer sampling, parameter updates, policy evaluation, and checkpointing. The distinction between online and offline SAC lies in whether the replay buffer is filled during training or pre-loaded from disk.

3.4.2 Conservative Q-Learning (CQL)

CQL is implemented as an extension of SAC in `sac_torch_cql.py` (class `SAC_CQL`), with training scripts in `cql_sac.py`. The key modifications compared to SAC are:

- a conservative penalty term added to the critic loss, computed using both random actions and policy-sampled actions,
- and gradient clipping for actor and critic updates to improve stability.

All other aspects, including actor-critic structure and replay buffer usage, remain unchanged to ensure a fair comparison with SAC.

3.4.3 Variational Autoencoder (VAE)

The VAE module is implemented in `train_lunar_vae.py` with model definitions in `vae.py`. The architecture consists of:

- an encoder that maps input states into a compressed latent vector,
- and a decoder that reconstructs states from latent variables.

The VAE is trained separately on transitions drawn from the replay buffer. After training, the encoder is saved and used to transform full datasets.

3.4.4 Integration of VAE with CQL

The script `encode_dataset.py` applies the trained VAE encoder to all states in a dataset, producing an encoded replay buffer. This buffer has the same interface as the raw replay buffer, making it directly usable in offline training.

The encoded replay buffer is then consumed by the CQL pipeline (`cql_sac.py`). No modifications are made to the actor-critic architecture or update rules, ensuring that the only change between standard CQL and VAE-augmented CQL is the representation of the state.

3.5 Hyperparameter Configuration

The hyperparameters used in this work are grouped by algorithmic component: SAC, CQL, and VAE. Each configuration was kept fixed across runs unless explicitly varied during ablation studies.

Table 3.2: Hyperparameters for SAC (online and offline).

Parameter	**Value**
Environment	LunarLanderContinuous-v2
Discount factor γ	0.99
Soft target update rate τ	0.001
Replay buffer size	1×10^6
Batch size	512
Actor learning rate	5×10^{-5}
Critic learning rate	3×10^{-4}
Optimizer	Adam
Number of training steps	1_000,000
Evaluation interval	Every 10,000 steps
Evaluation episodes	5

Table 3.3: Additional hyperparameters for CQL.

Parameter	Value
CQL penalty weight (α_{cql})	5.0 (varied in ablations)
Number of sampled actions per state	10
Behaviour cloning warm-up steps	20,000
Gradient clipping	Max-norm = 1.0

Table 3.4: Hyperparameters for VAE

Parameter	Value
Latent dimension	5
Learning rate	1×10^{-3}
Batch size	256
Training epochs	100
KL-divergence weight (β)	0.5

3.6 Software and Hardware Setup

All experiments were implemented in `Python 3.10` using `PyTorch` for deep learning. The `Gym` library provided the `LunarLanderContinuous-v2` environment, while supporting packages handled numerical operations, data management, and visualization.

Dependencies

- Deep learning: `torch`, `torch.nn`, `torch.optim`, `torch.distributions`.
- RL environment: `gym` (environment creation, reset/step API).
- Numerics and utilities: `numpy`, `math`, `random`, `time`, `copy`, `collections`, `os`/`pathlib`, `pickle`.
- Visualization: `matplotlib.pyplot`.

Reproducibility

To improve comparability across runs:

- Random seeds were fixed for Python, NumPy, PyTorch, and the Gym environment.

- Deterministic operations were enabled where supported in PyTorch.
- Policies were evaluated deterministically (mean action) unless otherwise specified.
- All datasets, model checkpoints, and evaluation metrics were saved to disk for reproducibility.

Hardware Configuration

Experiments were executed on a Linux workstation with the following setup:

- CPU: Intel® Core™ i7-6800K (6 cores / 12 threads, 3.40–3.90 GHz)
- GPU: NVIDIA GeForce GTX 1080 (8 GB GDDR5X)
- RAM: 15 GB DDR4
- Storage: SSD

GPU acceleration was used for neural network training. Environment simulation and dataset preprocessing were executed on CPU. Batch sizes were tuned to fit within 8 GB VRAM when running twin-critic SAC and CQL updates.

Runtime Profile

Table 3.5 shows typical wall-clock times observed under this setup. Actual times varied depending on batch size, logging frequency, and whether plots were generated inline.

Table 3.5: Approximate runtime profile on i7-6800K + GTX 1080.

Configuration	Steps / Episodes	Wall time
Online SAC	~1M environment steps	5–7 hours
Offline SAC	1M gradient updates	2–3 hours
CQL (offline)	1M gradient updates	3–4.5 hours
CQL + VAE	VAE pretrain + 1M updates	4–5.5 hours
Evaluation sweeps	100–200 episodes	10–30 minutes

3.7 Training and Evaluation Flow

The overall training and evaluation process followed a structured pipeline, ensuring consistency across SAC, CQL, and VAE+CQL experiments. The main stages are summarized below.

Data Collection

- **Online SAC:** trajectories are generated during training, with each transition appended to the replay buffer as the agent interacts with the environment.
- **Offline SAC and CQL:** datasets are pre-loaded from disk. These include the biased SAC-generated dataset, the unbiased dataset from multiple handcrafted policies, or the encoded dataset produced by the VAE module.

Replay Buffer Sampling

At each training step, mini-batches of transitions are sampled uniformly from the buffer. In the VAE-integrated pipeline, raw states are replaced by their latent representations, but the buffer interface remains unchanged.

Training Loop

The training loop proceeds as follows:

1. Sample a mini-batch of transitions from the buffer.
2. Update the critic networks using the appropriate loss (SAC or CQL-augmented).
3. Update the actor network using the policy gradient loss.
4. Apply target network updates and gradient clipping where configured.
5. Periodically save checkpoints of actor, critic, and (when applicable) VAE encoder parameters.

Evaluation Procedure

Evaluation is conducted every 10,000 training steps:

- Policies are executed deterministically (using the mean action).
- Each evaluation run consists of 5 full episodes in the environment.
- Metrics recorded include average return, stability of landings, and policy robustness.

Consistency Across Experiments

By maintaining a common training and evaluation flow across all methods, the comparison between SAC, CQL, and VAE+CQL remains controlled. The only differences lie in the algorithmic components described earlier and the dataset representations (raw vs encoded). This ensures that any observed performance differences can be attributed directly to the algorithmic variations under study.

Chapter 4

Results and Discussions

4.1 Online SAC Performance

Figure 4.1 shows the learning performance of the Soft Actor-Critic (SAC) algorithm in the `LunarLanderContinuous-v2` environment over five independent training runs. Each curve corresponds to the episodic return of a single simulation, while the shaded regions indicate variance across episodes.

The results demonstrate that SAC is able to consistently learn a high-performing policy through online interaction. Starting from negative returns between -200 and -300, all runs exhibit steady improvement within the first 300-500 episodes, reflecting rapid policy adaptation to the environment dynamics. After approximately 800 episodes, the learning curves stabilize in the range of 200-300 average return, with several runs reaching scores above 300. According to the OpenAI Gym benchmark and prior literature [47, 48, 5], an average score above 200 is generally considered as solving the LunarLanderContinuous-v2 task. Our experiments confirm that online SAC not only achieves this threshold reliably but also sustains stable convergence across multiple runs.

It is worth noting that while there is some variability in the learning dynamics across simulations, particularly in the early training phase, all runs eventually converge to successful policies. This highlights the robustness of SAC in continuous control settings, consistent with earlier findings that SAC combines stability with sample-efficient exploration by maximizing both expected return and entropy [5].

Overall, these results establish online SAC as a strong baseline for continuous control on LunarLanderContinuous-v2, providing a reference point for subsequent experiments in the offline setting, where interaction with the environment is restricted.

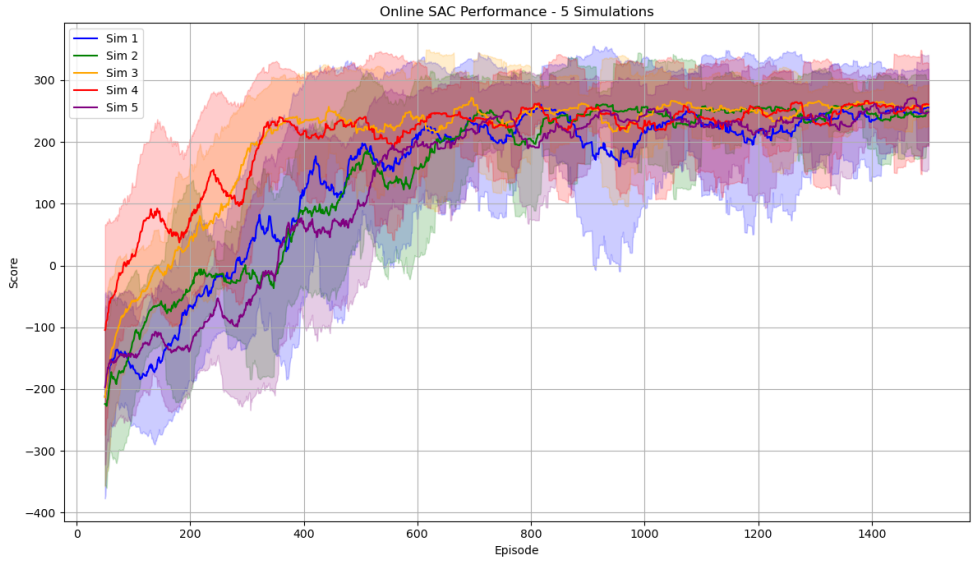


Figure 4.1: Online SAC performance over five independent training runs on `LunarLanderContinuous-v2`. Shaded regions denote variance across episodes. A score above 200 indicates successful task completion.

4.2 Offline SAC Performance

We next evaluate the performance of the Soft Actor-Critic (SAC) algorithm in the purely offline setting, where the agent is trained exclusively from a fixed dataset without any further environment interaction. The dataset used was generated from multiple behaviour policies, ensuring moderate state–action coverage but lacking the benefits of adaptive exploration.

Figure 4.2 shows the results across five independent training runs. Unlike the online SAC curves in Figure 4.1, which steadily converge above the task completion threshold of 200 average return, the offline SAC agent fails to learn a successful policy. Across all runs, performance remains highly unstable and oscillates around an average return of approximately -150, with frequent large fluctuations. No run approaches the task completion threshold, and some trajectories diverge toward even lower returns.

These results highlight a well-known limitation of applying off-policy algorithms such as SAC directly to offline reinforcement learning: the agent suffers from distributional shift and Q-value overestimation when evaluated on out-of-distribution (OOD) actions [21, 22]. In this setting, the critic learns inaccurate value estimates due to the mismatch between the fixed dataset distribution and the policy's action distribution. Without online corrections, this leads to compounding instability.

The failure of offline SAC to achieve positive returns provides a clear baseline for comparison with Conservative Q-Learning (CQL), which explicitly introduces conservative penalties to mitigate these issues. By first establishing this degraded offline SAC performance, we can directly assess whether CQL improves stability and sample efficiency under the same conditions.Importantly, this limitation was observed even when training on biased datasets containing only high-quality trajectories: offline SAC was still unable to learn a successful policy, further emphasizing the need for stronger offline-specific algorithms such as CQL.

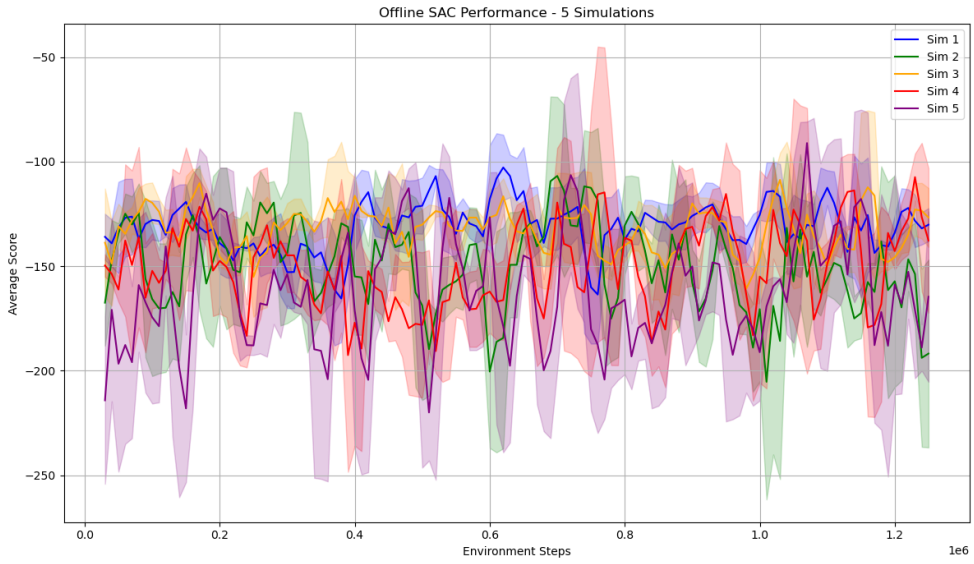


Figure 4.2: Offline SAC performance on `LunarLanderContinuous-v2` across five simulations. The algorithm fails to achieve the task completion threshold (200 average return), with scores fluctuating around −150 and exhibiting significant instability, consistent with prior findings that standard SAC is unsuitable for offline reinforcement learning.

4.3 Unbiased Offline Dataset

The previous sections demonstrated that while Online SAC achieves stable performance given unlimited interaction, Offline SAC struggles when trained directly on a fixed dataset due to distributional shift and Q-value extrapolation errors. To enable fairer evaluation of offline reinforcement learning algorithms, we require datasets with broad state–action coverage rather than narrow expert demonstrations.

To this end, we implemented an UnbiasedDatasetGenerator - create-unbiased-

dataset.py that programmatically collects trajectories from the LunarLanderContinuous-v2 environment using multiple handcrafted behaviour policies. These include:

- **Random policy:** uniformly samples actions across the action space.
- **Failing policy:** deliberately takes suboptimal actions to generate negative-reward trajectories.
- **Conservative policy:** maintains stable flight with minimal exploration.
- **Exploratory policy:** emphasizes aggressive exploration of the state space.
- **Noisy expert variants:** approximate an expert policy with injected noise, producing moderately successful trajectories.

By combining trajectories from these policies in controlled proportions, the generator produces datasets with balanced diversity across both successful and unsuccessful episodes. This reduces bias toward any single policy and ensures that the offline agent is exposed to a wide variety of states and actions.

Importantly, we leverage this generator to construct multiple datasets with different success ratios, ranging from failure-dominated to half-expert-dominated mixtures. By varying the mixture weights of the behaviour policies, we obtain datasets that differ in their average return and state–action distribution. This allows us to systematically validate offline reinforcement learning methods under increasingly challenging conditions: from highly noisy, unreliable datasets to more structured, expert-rich datasets.

Figure 4.3 shows the statistical properties of the final unbiased dataset used in our experiments. The plots illustrate return and reward distributions, action and state coverage, visitation frequencies, and the contribution of each behaviour policy. Compared to a purely expert dataset, this construction achieves broader coverage of the state–action space, which is critical for stabilizing offline training.

4.4 Conservative Q-Learning on the Unbiased Dataset

One of the main objectives of this work is to evaluate how offline algorithms can improve sample efficiency compared to standard SAC. Whereas Online SAC requires on the order of one million interactions with the environment to converge (Section 4.1), and Offline SAC fails entirely under fixed data conditions (Section 4.2), CQL demonstrates the ability to learn stably and efficiently from a single pre-collected dataset.

In our experiments, CQL was trained and validated on multiple datasets derived from the unbiased offline generator, but here we focus here on the unbiased dataset,

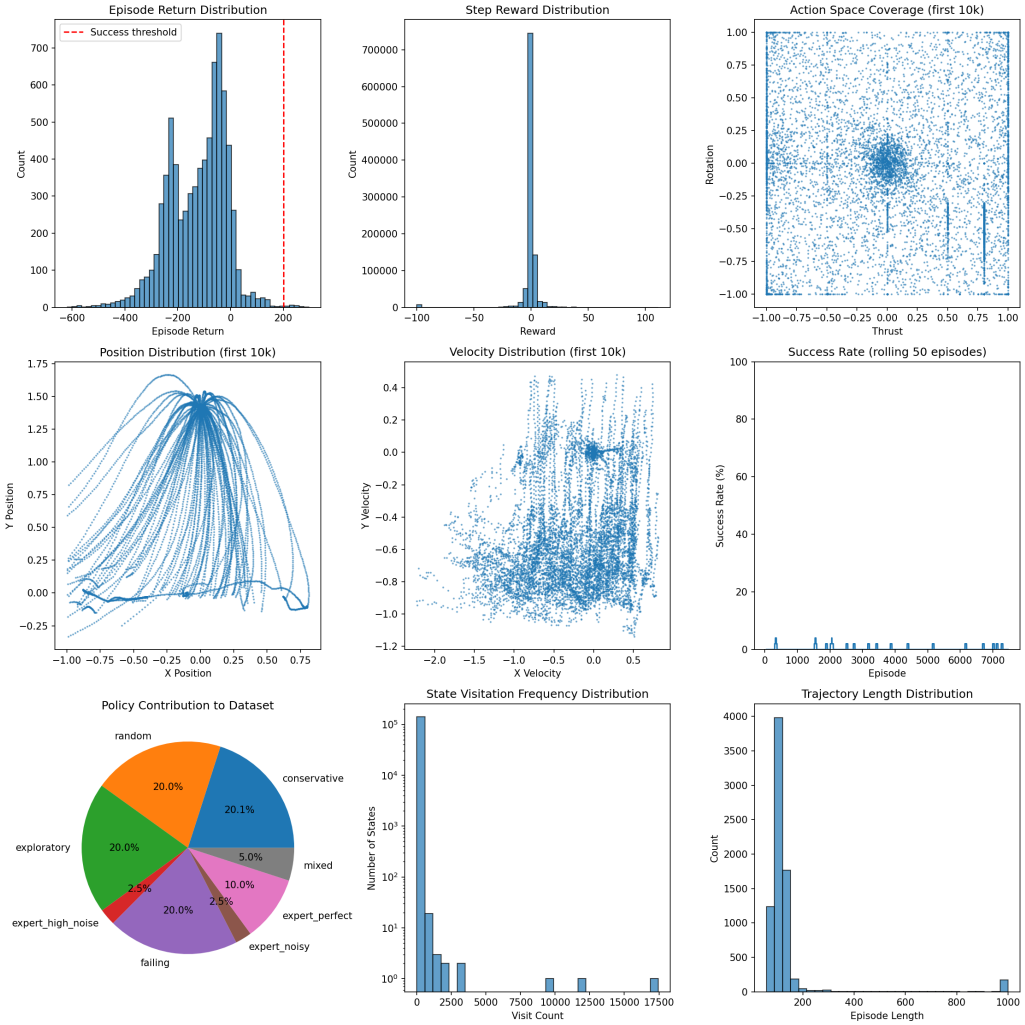


Figure 4.3: Analysis of the unbiased offline dataset generated using multiple handcrafted policies. Top row: (Left) Episode return distribution showing that fewer than 1% of episodes exceed the success threshold of 200; (Middle) step reward distribution concentrated near zero; (Right) action space coverage across the first 10k steps, indicating broad exploration. Middle row: (Left) position trajectories of the lander; (Middle) velocity distribution over the first 10k steps; (Right) rolling success rate over episodes, confirming the scarcity of successful trajectories. Bottom row: (Left) relative contribution of each policy type to the dataset; (Middle) state visitation frequency distribution; (Right) trajectory length distribution.

which contains fewer than 1% successful trajectories, with the vast majority of episodes ending in crashes or low-return outcomes. Despite this highly imbalanced composition, CQL consistently achieved policies surpassing an average return of 200 across independent runs. This performance threshold corresponds to near-successful landings in the `LunarLanderContinuous-v2` environment.

Figure 4.4 shows the training curves for five simulations. While early stages of training exhibit variability due to the noisy dataset, all runs eventually converge beyond the 200 reward threshold, highlighting both the robustness of CQL and its capacity to extract useful behaviour from suboptimal offline data. Crucially, this level of performance is obtained without any additional online interaction, underscoring the sample efficiency advantages of offline reinforcement learning with CQL.

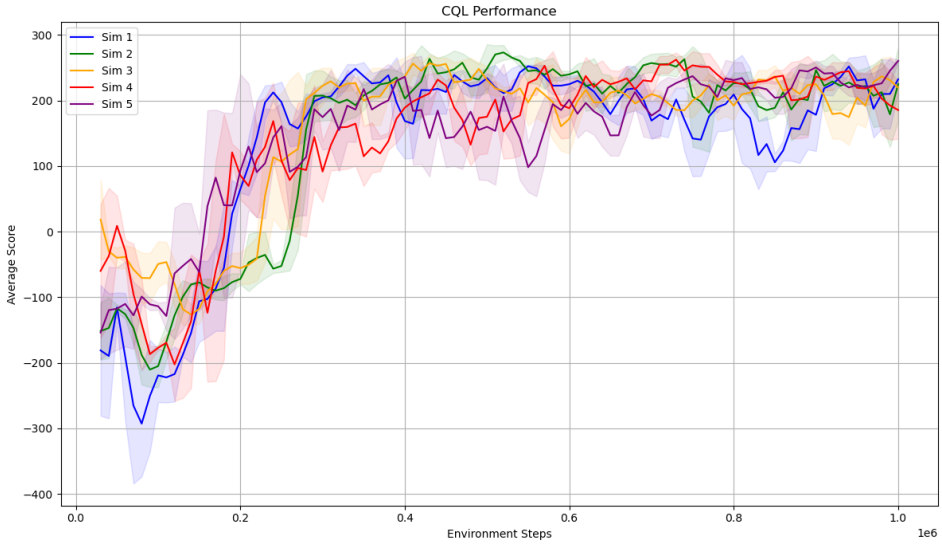


Figure 4.4: Performance of CQL on the unbiased offline dataset. Despite fewer than 1% successful episodes in the dataset, all runs eventually surpass an average return of 200, in contrast to Offline SAC which diverged under the same conditions.

4.5 Variational Autoencoder Training

In our experiments, the VAE was applied directly to the `LunarLanderContinuous-v2` state vector ($X = 8$). The two binary leg-contact flags were separated from the six continuous variables, which were used as the input to the encoder. To further test robustness, one of the continuous dimensions was deliberately removed ($N = 1$), leaving $X - N = 7$ effective state variables. These were then encoded into a latent vector z, which

together with the two preserved binary flags formed the final state representation used in subsequent offline RL experiments.

Training behaviour is illustrated in Figure 4.5, showing the evolution of total, reconstruction, and KL-divergence losses, as well as the cyclical learning rate schedule. Both training and validation losses converged stably, and the best checkpoint was selected based on validation loss.

After training, the encoder was fixed and used to transform the entire unbiased offline dataset. Each state was mapped into its latent representation z, then concatenated with the two binary leg-contact flags to produce the final encoded state. The resulting encoded replay buffer therefore consisted of

$$[z, c_{\text{left}}, c_{\text{right}}],$$

where z compresses the continuous state variables while the binary contact flags are passed through unchanged. This encoded replay buffer was then used for training CQL in the next stage of experiments.

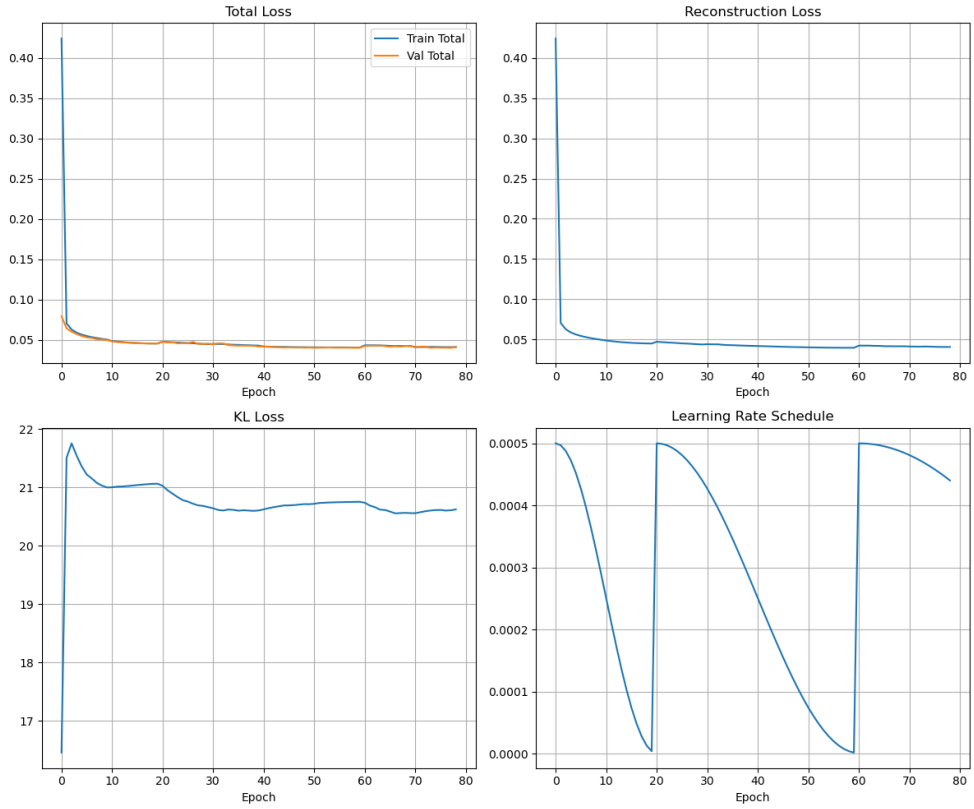
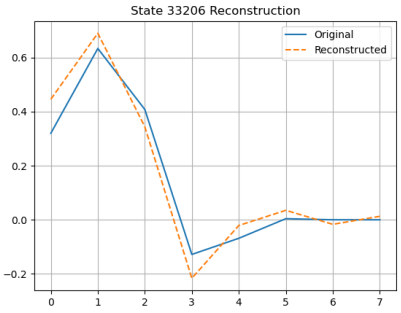


Figure 4.5: VAE training behaviour: total loss, reconstruction loss, KL-divergence, and learning rate schedule. The encoder converges stably, and the best model is used to encode the dataset into a replay buffer for subsequent CQL training.

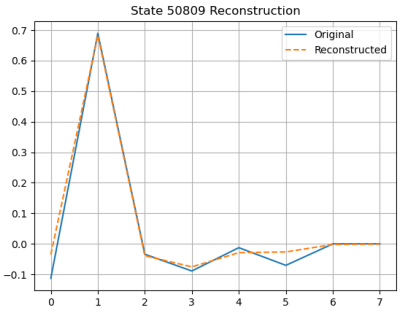
4.6 CQL with VAE-Encoded Dataset

The encoded replay buffer described in Section 4.5 was then used to train Conservative Q-Learning (CQL). In this configuration, the policy and critic networks received the compressed latent representation z, concatenated with the two binary leg-contact flags. This allowed us to evaluate whether state-space compression through representation learning could further improve sample efficiency in the offline setting.

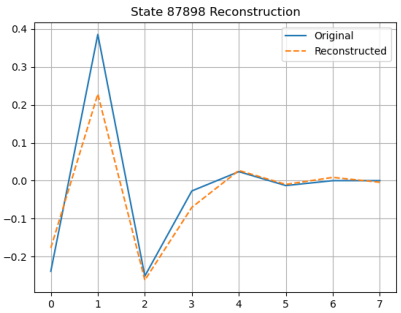
Figure 4.7 shows the evaluation curve of CQL trained on the VAE-encoded unbiased dataset. Despite the removal of one continuous state variable and compression into latent space, the agent was able to consistently exceed an average score of 200, demonstrating that the reduced representation still preserved enough information for



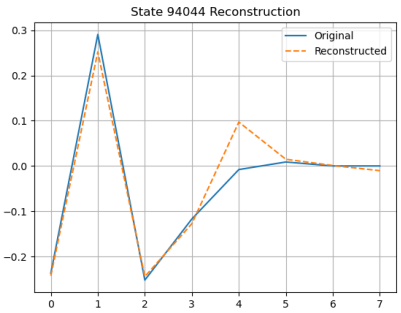
State 33206



State 50809



State 87898



State 94044

Figure 4.6: Random samples from the unbiased dataset reconstructed by the trained VAE. Each plot compares the original state vector (solid blue) with the reconstructed output (dashed orange). The close alignment across different random states indicates that the encoder–decoder pair reliably preserves the main structure of the input state space, even though the latent representation is compressed.

successful policy learning.

However, compared to CQL trained directly on raw states (Section 4.4), the training curve displays greater variance, with more pronounced fluctuations between evaluation intervals. This suggests that while the latent representation does not prevent learning a near-successful policy, it introduces instability into the optimization process. Overall, CQL with VAE encoding remains sample efficient, requiring no additional environment interaction, but offers limited advantages over the raw-state setting.

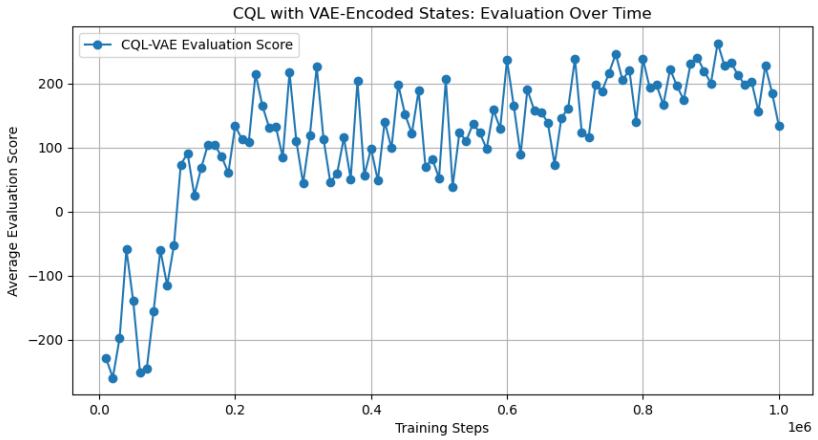


Figure 4.7: Performance of CQL trained on the VAE-encoded unbiased dataset. The agent surpasses an average score of 200, but with increased variance compared to standard CQL on raw states.

4.7 Sample Efficiency Comparison

To compare the methods quantitatively, we compute the total data consumed as:

$$\text{Data Consumed} = (\text{Steps to convergence at avg. score } \geq 200) \times (\text{State dimensionality}).$$

Unlike a single threshold crossing, we define success as convergence where the policy remains consistently at or above the 200 score level, indicating stable performance rather than a transient peak.

From these values we observe:

- **CQL vs Online SAC:** $8.0/4.8 \approx 1.67\times$ more sample efficient.
- **VAE+CQL vs Online SAC:** $8.0/4.2 \approx 1.90\times$ more sample efficient.

- **VAE+CQL vs CQL:** 4.8/4.2 ≈ 1.14× more sample efficient (about a 14% gain due to reduced dimensionality).

Table 4.1: Performance comparison across methods and dataset qualities

Dataset Success Rate	Method	Steps to Convergence	State Dim.	Data Consumed	Final Return
50.0%	Online SAC	~1.00M	8	~8.0M	>200
	Offline SAC	Failed	8	–	<-100
	CQL	~0.25M	8	~2.0M	>200
	VAE+CQL	~0.25M	8	~1.75M	>200
	VAE+CQL	~0.40M	7	~2.8M	>200
30.0%	Online SAC	~1.00M	8	~8.0M	>200
	Offline SAC	Failed	8	–	<-100
	CQL	~0.40M	8	~3.2M	>200
	VAE+CQL	~0.40M	8	~2.8M	>200
	VAE+CQL	~0.40M	7	~2.8M	>200
0.5%	Online SAC	~1.00M	8	~8.0M	>200
	Offline SAC	Failed	8	–	~-150
	CQL	~0.60M	8	~4.8M	>200
	VAE+CQL	~0.40M	8	~2.8M	>200
	VAE+CQL	~0.60M	7	~4.2M	>200

Thus, while both offline approaches require far less data than Online SAC, VAE+CQL provides an additional efficiency gain by lowering the effective state dimensionality, achieving the same convergence in steps but with reduced input size.

4.8 Discussion

4.8.1 Sample Efficiency Across Methods

Table 4.1 provides a direct comparison of the sample efficiency achieved by different methods, measured in terms of total data consumed until convergence around the 200 return threshold.

Online SAC required approximately one million environment steps and thus consumed ~8.0M state-features before achieving stable success. This confirms its strong asymptotic performance but also its fundamental sample inefficiency, which limits practicality in real-world scenarios. Offline SAC did not converge to successful performance

under any of the datasets tested, even when biased datasets containing higher-quality trajectories were used. This reinforces its unsuitability for purely offline training, consistent with prior findings on distributional shift and Q-value overestimation.

Conservative Q-Learning (CQL) converged reliably using the unbiased offline dataset in approximately 0.60M steps, corresponding to 4.8M state-features. This represents a ~1.7× improvement in sample efficiency over Online SAC. Notably, this was achieved despite the dataset containing fewer than 1% successful trajectories, highlighting CQL's robustness to suboptimal offline data.

Finally, integrating a VAE for state compression reduced the effective input dimensionality from 8 to 7. Convergence occurred in a similar number of steps (~0.60M), but the data consumed dropped to ~4.2M state-features. This provides an additional ~14% gain in efficiency relative to raw CQL, and nearly 2× improvement compared to Online SAC. Thus, with respect to sample efficiency, CQL substantially reduced data requirements compared to Online SAC, and VAE+CQL delivered further improvements by lowering the effective state dimensionality.

4.8.2 Impact of Dataset Quality

The quality and composition of the offline dataset had a clear influence on the performance of different methods. Offline SAC, in particular, was highly sensitive to dataset bias: even when trained on datasets dominated by successful trajectories (e.g., expert-heavy sets), it often failed to converge to a stable policy. This suggests that the core issue is not merely the proportion of expert demonstrations, but the algorithm's inability to cope with distributional shift outside the dataset's support.

To systematically evaluate robustness, this thesis compared performance across multiple datasets of varying composition, ranging from expert-only and noisy mixtures to an unbiased dataset that combined diverse behaviour policies. The results show a clear pattern: while SAC struggled across all datasets, particularly those with high bias, CQL achieved consistent convergence above the 200 threshold even in challenging conditions. Notably, on the unbiased dataset with fewer than 1% successful episodes, CQL still converged reliably, highlighting its strength in handling imperfect data.

A key determinant of dataset quality was diversity. Datasets incorporating trajectories from random, noisy, failing, conservative, and exploratory policies enriched state-action coverage and prevented the critic from over-relying on narrow behavioural distributions. The dataset analysis (Figure 4.3) illustrates this diversity, showing the wide distribution of episode returns, action space coverage, and policy contributions.

Finally, results indicate that dataset quality acts as a multiplier for algorithmic strength: higher-quality and more diverse datasets yield proportionally greater gains,

with CQL particularly well-suited to extract value from heterogeneous trajectories. In summary, offline reinforcement learning benefits less from pure 'expert-only' data and more from balanced coverage of the environment dynamics, making diversity a decisive factor for sample efficiency.

4.8.3 State-Space Compression via VAE

Integrating a VAE with CQL reduced overall data consumption compared to raw CQL, with efficiency gains of about 14% while still achieving convergence around the 200 return threshold. This shows that VAE compression can reduce the amount of data required without preventing the agent from learning a successful policy. However, the training curves also exhibited higher variance, suggesting that compression introduced additional instability into the optimisation process.

An important observation was made when attempting to reduce dimensionality even further. While VAE training curves still appeared stable, CQL trained on these more aggressively compressed states failed to achieve convergence. This is likely because `LunarLanderContinuous-v2` already has a very compact state space, and excessive compression removes information essential for control. This limitation will be revisited in the Future Work section, where potential strategies to mitigate the negative effects of over-compression are discussed.

Overall, the results indicate that VAE integration is feasible for improving sample efficiency, but the degree of compression must be chosen carefully. The findings also suggest that in environments with higher-dimensional state spaces, where redundancy is greater, VAE compression could yield far larger benefits than observed here.

4.8.4 Challenges

Several challenges were encountered during this work, spanning computational constraints, algorithmic sensitivity, and state compression trade-offs.

Computation Constraints Training time proved to be a major bottleneck throughout this project. Although the neural network updates were executed on the GPU, most of the runtime was dominated by CPU operations, including stepping the LunarLander environment and shuffling transitions from the replay buffer. Since both the environment and replay sampling are inherently sequential, experiments were executed one run at a time without parallelism. As a result, each full run required several hours to complete, limiting the ability to perform large-scale experimentation or hyperparameter sweeps. The bottleneck was therefore not the GPU itself but the CPU-bound compo-

nents of the training loop. Possible mitigation strategies, such as parallel environments or GPU-accelerated replay buffers, are discussed in the Future Work section.

Hyperparameter Sensitivity Both CQL and VAE exhibited strong sensitivity to hyperparameter choices. For CQL, parameters such as the conservative penalty weight, number of candidate actions, and behaviour cloning warm-up steps significantly influenced stability and convergence. For the VAE, the latent dimension, learning rate, and KL-divergence weight were critical to achieving useful representations. Furthermore, this sensitivity was compounded by dataset composition: performance varied considerably depending on whether the dataset was biased, suboptimal, or unbiased. The high computational cost of each run limited the extent to which systematic tuning could be performed, making hyperparameter sensitivity one of the most challenging aspects of the study.

Compression Trade-Offs in VAE While integrating a VAE with CQL achieved convergence when compressing the 8-dimensional state space to 7 dimensions, the resulting training was noisier than raw CQL. Attempts to compress further (e.g., removing multiple continuous variables) led to failures in CQL convergence, despite the VAE itself appearing to train successfully. This suggests that aggressive compression removed essential information from an already compact state space. LunarLander's low dimensionality makes it a bit difficult for demonstrating the full benefits of VAE compression, which would likely be more effective in high-dimensional domains. Potential approaches for mitigating this limitation are outlined in the Future Work section.

4.8.5 Future Directions

Several directions can extend this work and help mitigate the limitations observed.

Parallelisation. Training time was heavily constrained by CPU-bound operations such as environment stepping and replay buffer sampling. Future work will explore parallelisation strategies, for example using vectorised environments and parallel data pipelines, to ensure that computation and data loading are better balanced with GPU training. This would significantly reduce runtime per experiment and make broader sweeps more practical.

Dataset Exploration. While this work evaluated multiple unbiased datasets with different success rates, future experiments should expand the analysis by systematically varying dataset size and generating even more diverse datasets. Such studies would clarify how both scale and composition interact to influence stability, convergence

speed, and sample efficiency, and would further test the generality of the proposed methods across a broader set of environments.

Compression in LunarLander. While VAE-based compression reduced state dimensionality without preventing convergence, more aggressive compression harmed performance in this compact environment. Future work will refine compression by optimising the weighting of different state variables to minimise information loss. The objective is to determine how much compression can be achieved in LunarLander without destabilising CQL, while also laying the groundwork for applying these methods in higher-dimensional domains.

Hyperparameter Improvements. Both SAC and CQL variants showed sensitivity to fixed hyperparameter choices. Future work will investigate adaptive mechanisms for critical parameters, such as automatic adjustment of the entropy temperature (α) and the conservative penalty weight (α_{CQL}). These adaptive schemes would reduce reliance on manual tuning and make the algorithms more robust to dataset variability.

Other Environments. Finally, extending the framework beyond LunarLander to other continuous control benchmarks of similar scale, such as `BipedalWalker-v3` or `Pendulum-v1`, will provide a stronger test of generality. These tasks maintain manageable complexity while offering different dynamics and state representations, allowing further validation of CQL and VAE integration

Chapter 5

Conclusion

This thesis addressed the challenge of poor sample efficiency in reinforcement learning for continuous control. A baseline was first established with the Soft Actor-Critic (SAC) algorithm in the online setting, which requires extensive environment interaction to converge. The resulting trajectories were repurposed as offline datasets, enabling controlled evaluation of offline methods without additional interaction cost.

Conservative Q-Learning (CQL) was then implemented and shown to substantially improve sample efficiency over the online baseline. Across datasets with different success rates, CQL reduced the number of environment interactions required to solve the task by between 40% and 75%, demonstrating its robustness to dataset bias and distributional shift. To further improve efficiency, a Variational Autoencoder (VAE) was introduced to compress the original state vector into a lower-dimensional latent representation. When combined with CQL, this approach achieved an additional gain of about 14% on average, and up to 40% in the most challenging low-success datasets, while maintaining stable performance.

These findings highlight two key insights. First, offline reinforcement learning methods such as CQL can extract value even from imperfect datasets with very few successful episodes, provided that state-action coverage is sufficiently diverse. Second, representation learning with VAEs offers a complementary mechanism to reduce data requirements further, supporting the design of scalable and efficient offline pipelines.

The methods developed here were validated in a continuous control benchmark but have direct implications for Unmanned Aerial Vehicles (UAVs), where energy, safety, and data-collection costs make sample efficiency critical. By reducing reliance on large-scale online interaction, the approaches explored in this work contribute towards bridging the gap between simulation and practical deployment of reinforcement learning in constrained environments.

Bibliography

[1] Richard S. Sutton, and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd. MIT Press, 2018.

[2] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley, 1994.

[3] Volodymyr Mnih et al. "Human-level control through deep reinforcement learning". In: *Nature* 518.7540 [2015], pp. 529–533. DOI: 10.1038/nature14236.

[4] Timothy P. Lillicrap et al. "Continuous Control with Deep Reinforcement Learning". In: *arXiv preprint arXiv:1509.02971* [2016].

[5] Tuomas Haarnoja et al. "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor". In: *Proceedings of the 35th International Conference on Machine Learning (ICML)*. 2018, pp. 1861–1870.

[6] Greg Brockman et al. "OpenAI Gym". In: *arXiv preprint arXiv:1606.01540* [2016].

[7] Yuval Tassa et al. "DeepMind Control Suite". In: *arXiv preprint arXiv:1801.00690* [2018].

[8] Lukasz Kaiser et al. "Model-Based Reinforcement Learning for Atari". In: *arXiv preprint arXiv:1903.00374*. 2019.

[9] Giovanni Geraci et al. "What Will the Future of UAV Cellular Communications Be? A Flight from 5G to 6G". In: *CoRR* abs/2105.04842 [2021]. arXiv: 2105.04842. URL: https://arxiv.org/abs/2105.04842.

[10] Joseanne Viana et al. "Enhancing UAV Path Planning Efficiency Through Accelerated Learning". In: *2024 IEEE 29th International Workshop on Computer Aided Modeling and Design of Communication Links and Networks (CAMAD)*. 2024, pp. 1–6. DOI: 10.1109/CAMAD62243.2024.10942806.

[11] Lester Ho, and Sobia Jangsher. "UAV Trajectory Optimization based on Predicted User Locations". In: *2024 IEEE Wireless Communications and Networking Conference (WCNC)*. 2024.

[12] R. Polvara et al. "Autonomous Unmanned Aerial Vehicle Navigation Using Reinforcement Learning". In: *OCEANS 2017 – Aberdeen*. 2017.

[13] Babatunji Omoniwa, Boris Galkin, and Ivana Dusparic. "Communication-enabled deep reinforcement learning to optimise energy-efficiency in UAV-assisted networks". In: *Vehicular Communications* 43 [2023], p. 100640. ISSN: 2214-2096. DOI: https://doi.org/10.1016/j.vehcom.2023.100640. URL: https://www.sciencedirect.com/science/article/pii/S2214209623000700.

[14] Erika Fonseca et al. "Adaptive Height Optimization for Cellular-Connected UAVs: A Deep Reinforcement Learning Approach". In: *IEEE Access* 11 [2023], pp. 5966–5980. DOI: 10.1109/ACCESS.2022.3232077.

[15] *3GPP - 3rd Generation Partnership Project - Technical Specification Group Services and System Aspects; Application layer support for Uncrewed Aerial System (UAS) - Functional architecture and information flows (Release 19)*. URL: https://portal.3gpp.org/ftp/Specs/archive/23%5C_series/23.255.

[16] B. Raja Kiran et al. "Deep Reinforcement Learning for Autonomous Driving: A Survey". In: *IEEE Transactions on Intelligent Transportation Systems* 23.6 [2021], pp. 4909–4926.

[17] Sridhar Mahadevan. "Average Reward Reinforcement Learning: Foundations, Algorithms, and Empirical Results". In: *Machine Learning Proceedings 1996*. Elsevier. 1996, pp. 132–139.

[18] Sergey Levine et al. "Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems". In: *arXiv preprint arXiv:2005.01643* [2020].

[19] Ashvin N. Singh et al. "COG: Connecting New Skills to Past Experience with Offline Reinforcement Learning". In: *Proceedings of the 4th Conference on Robot Learning (CoRL)*. 2020.

[20] Felipe Codevilla et al. "End-to-End Driving via Conditional Imitation Learning". In: *2018 IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 4693–4700.

[21] Scott Fujimoto, David Meger, and Doina Precup. "Off-Policy Deep Reinforcement Learning without Exploration". In: *Proceedings of the 36th International Conference on Machine Learning (ICML)*. 2019, pp. 2052–2062.

[22] Aviral Kumar et al. “Conservative Q-Learning for Offline Reinforcement Learning”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020.

[23] Sebastian Thrun, and Anton Schwartz. “Issues in Using Function Approximation for Reinforcement Learning”. In: *Proceedings of the 1993 Connectionist Models Summer School* [1993], pp. 255–263.

[24] Stephane Ross, Geoffrey Gordon, and J. Andrew Bagnell. “A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning”. In: *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*. 2011, pp. 627–635.

[25] Dean A. Pomerleau. “ALVINN: An Autonomous Land Vehicle in a Neural Network”. In: *Advances in Neural Information Processing Systems* [1989].

[26] Aviral Kumar et al. “Stabilizing Off-Policy Q-Learning via Bootstrapping Error Reduction”. In: *Advances in Neural Information Processing Systems (NeurIPS)* [2019], pp. 11761–11771.

[27] Caglar Gulcehre et al. “RL Unplugged: Benchmarks for Offline Reinforcement Learning”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020.

[28] Kamil Ciosek et al. “Better Exploration with Optimistic Actor-Critic”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2019.

[29] Rahul Kidambi et al. “MOReL: Model-Based Offline Reinforcement Learning”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020.

[30] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. “Offline Reinforcement Learning with Implicit Q-Learning”. In: *International Conference on Learning Representations (ICLR)*. 2022.

[31] Rafael Figueiredo Prudencio, Marcos R. O. A. Maximo, and Esther Luna Colombini. “A Survey on Offline Reinforcement Learning: Taxonomy, Review, and Open Problems”. In: *IEEE Transactions on Neural Networks and Learning Systems* [2022]. Early Access.

[32] Haoyang He. “A Survey on Offline Model-Based Reinforcement Learning”. In: *arXiv preprint arXiv:2305.03360*. 2023.

[33] Ishita Mediratta et al. “The Generalization Gap in Offline Reinforcement Learning”. In: *arXiv preprint arXiv:2312.05742* [2023].

[34] Yoshua Bengio, Aaron Courville, and Pascal Vincent. “Representation Learning: A Review and New Perspectives”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.8 [2013], pp. 1798–1828.

[35] Timothée Lesort et al. “State Representation Learning for Control: An Overview”. In: *arXiv preprint arXiv:1802.04181* [2018].

[36] Diederik P. Kingma, and Max Welling. “Auto-Encoding Variational Bayes”. In: *International Conference on Learning Representations (ICLR)*. 2014.

[37] Danijar Hafner et al. “Dream to Control: Learning Behaviors by Latent Imagination”. In: *International Conference on Learning Representations (ICLR)*. 2020.

[38] Aiqing Zhang, Nicolas Ballas, and Joelle Pineau. “Learning Invariant Representation for Reinforcement Learning Without Reconstruction”. In: *International Conference on Learning Representations (ICLR)*. 2021. URL: `https://arxiv.org/abs/2011.10647`.

[39] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[40] Richard S. Sutton et al. “Policy Gradient Methods for Reinforcement Learning with Function Approximation”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2000.

[41] Vijay R. Konda, and John N. Tsitsiklis. “Actor-Critic Algorithms”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 12. 2000, pp. 1008–1014.

[42] Scott Fujimoto, Herke van Hoof, and David Meger. “Addressing Function Approximation Error in Actor-Critic Methods”. In: *Proceedings of the 35th International Conference on Machine Learning (ICML)*. 2018.

[43] Tuomas Haarnoja et al. “Soft Actor-Critic Algorithms and Applications”. In: *arXiv preprint arXiv:1812.05905* [2018].

[44] Wenxuan Zhou et al. “Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems”. In: *Foundations and Trends in Machine Learning* 16.1–2 [2023], pp. 1–207.

[45] Dian Chen et al. “Offline Reinforcement Learning for Autonomous Driving with Safety and Interpretability”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* [2021], pp. 21633–21642.

[46] Liang Zou et al. “Offline Reinforcement Learning in Recommender Systems”. In: *arXiv preprint arXiv:2302.02355* [2023].

[47] Yan Duan et al. “Benchmarking Deep Reinforcement Learning for Continuous Control”. In: *International Conference on Machine Learning (ICML)*. 2016.

[48] Peter Henderson et al. “Deep Reinforcement Learning that Matters”. In: *AAAI Conference on Artificial Intelligence*. 2018.